MATH300: Mathematical Computing · Fall 2026

Project: Compressing a Video with the (Incremental) SVD

Groups must be formed, and project titles submitted on Canvas, by **October 26, 2026**. The project has two graded deliverables and an oral exam: a 3–5 page report for Project 2 (due **November 20, 2026**), a poster for the final project (due **December 11, 2026**), and an oral exam held between November 30 and December 17, 2026, 3:30 pm. Read the general rules below before you start; they apply throughout the project.

General remarks

- Projects are set for a group of **two or three** students.
- Form your group by **October 26, 2026**, and submit your project title as a group assignment on **Canvas** by the same date.
- Every group member must carry an **equal portion** of the coding and of the report and poster preparation.
- The scope can be adjusted to the number of participants, their interests and prior knowledge.
- Only **Python** may be used for programming.
- Implementation, documentation, validation and demonstration examples are all part of the project.
- Within these rules you are free to choose the implementation, code structure and exact content. You can discuss this with me at any time.

Version control and GitHub

Each group keeps **one GitHub repository for the whole project** (Project 2 and the final project), and every member commits from their own GitHub account.

- Use git from the very beginning. Commits should be small, consistent and clearly described. Avoid large commits.
- Each group member uploads their own code using **their own GitHub account**. Commits from a shared or borrowed account will not be credited to the other members.
- The repository name is up to your group.
- If the repository is private, add my GitHub account `say-yas` as a collaborator.
- The repository must contain a README describing the project, how to install and run the code, and the AI-use statement (see below).

Use of AI tools

AI-generated code is not accepted in any form. Code produced by a chat assistant or code generator (e.g. ChatGPT, Claude, Gemini) and copied, pasted or adapted into your project will be treated as work that is not your own.

- **Permitted:** inline coding assistance such as GitHub Copilot autocompletion while you write the code yourself, **provided its use is acknowledged**.
- **Acknowledgement:** include an AI-use statement in the README and in the report that names the tool(s) used and the parts of the code (files or functions) where they assisted. If no AI tools were used, state this explicitly.
- **Reports and posters must not be generated by AI** in any form: their text, figures and layout must be your own work. If AI tools were used for any other non-code purpose (e.g. checking grammar or searching the literature), describe this in the same statement.
- You are responsible for every line in your repository. In the oral exam you must be able to explain and modify any part of the code you submitted.

Deliverables and deadlines

Submit each deliverable following the details provided on the **Canvas** page. The written report and poster are handed in on **Crowdmark**, and the link to your GitHub repository is submitted separately on **Canvas**.

Deliverable	**Due**	**Submission**	**Notes**
Group and project title	Monday, October 26, 2026	Group assignment on Canvas	Group of two or three members; submit the project title.
Project 2 report	Friday, November 20, 2026	Report (PDF) on Crowdmark; repository link on Canvas	3–5 pages; the page limit **includes** references, figures and appendices.
Final project poster	Friday, December 11, 2026	Poster (PDF) on Crowdmark; repository link on Canvas	36 inches high by 48 inches wide (landscape). Target audience: undergraduate classmates with a basic background in math, physics or engineering.
Oral exam	Between November 30 and December 17, 2026, 3:30 pm	Individual meeting; details discussed in class	30 minutes per student. Your poster must already be submitted on Crowdmark before your exam. The last possible slot ends December 17 at 3:30 pm.

Oral exam

The oral exam is individual and lasts 30 minutes per student. It takes place between November 30 and December 17, 2026, 3:30 pm. The oral exams and how to arrange them will be discussed in class before the Thanksgiving break.

Before your oral exam, your poster must also be submitted on **Crowdmark**; that is, you must have finished both Project 2 and the final project. In the meeting you will:

- Present your results and algorithms.
- Possibly live-code and run parts of your algorithms to generate outputs on the spot.
- Answer questions about the theory, your implementation and your results. Each member should be able to explain the whole project, not only their own parts.

Bring a laptop with the repository cloned and a working environment, so that the code runs without additional setup.

Grading

Component	Points
Group and project title (on time)	10
Project 2 report	100
Final project poster	100
Oral exam (individual)	50
Bonus: extra steps beyond the required tasks	up to 10

The Project 2 report and the poster are each graded out of 100. What matters most is:

- **Coherence:** a clear story from the question, through the methods, to the results and conclusions.
- **Correctness:** of the mathematics, the code and the results.
- **Depth of analysis and discussion:** explain what your results mean, why they look the way they do, and what their limitations are, rather than only showing them.
- **Professional presentation:** figures with labeled axes and readable font sizes, captions and legends, a clean and readable layout, and proper references.

A complete, correct and well-discussed solution of the required tasks can earn full marks. Tasks marked **(Bonus)** and the suggestions under "Going further" are optional; exploring such extra steps can earn **up to 10 bonus points** in total.

Requirements for the report and the poster

Both the report and the poster must contain a clear description of the project, references, properly labeled figures and an author-contributions statement.

Content

- A clear and concise description of the problem, the methods and the results.
- The AI-use statement described above.
- The report should not contain any significant amount of code; refer to the repository instead.

Poster

- The poster must be **self-contained**: a reader should be able to follow it without the report, the code or a spoken explanation.
- You may be selective in what you show and how you narrate your story, so that an undergraduate audience with a basic background in math, physics or engineering can follow it.

- You may choose your own title for the poster; be creative.

References

- Include a list of references at the end of the report and the poster.
- Cite sources clearly where they are used in the text (papers, books, software, code repositories and data sets).
- Use one consistent citation style throughout.

Figures

- Every plot must have labeled x and y axes (and z axis for 3D plots or color bars), including units where applicable.
- Axis labels, tick labels and legends must have a font size similar to the surrounding text, so they remain readable without zooming.
- Each figure needs a legend when it shows more than one data series, and a caption that states what is shown.
- Use log scales where appropriate (e.g. for convergence plots) and say so in the caption.

Author contributions

Each group must include an author-contributions statement in the report and the README that states:

- Who did what: which parts of the code, experiments, analysis, writing and poster each member worked on.
- To what extent: the approximate share of each member in each part (e.g. 50 % / 50 %).
- Who took the lead in which parts.

The coding and the report and poster preparation must be shared equally between the members. The commit history should be consistent with the stated contributions.

Codebase requirements

- Well structured and documented, both in the code (docstrings, comments) and in the repository (README).
- Validated: show that your implementation produces correct results, e.g. on examples with known solutions.

Project description

Goal

A video is a sequence of frames, and consecutive frames are usually very similar. In this project you will use the singular value decomposition (SVD) to compress a short video: you will approx-

imate all its frames by a few basis images. First you compute the SVD of the whole video at once with NumPy; then you implement the *incremental SVD* described in Section 2.2 of Kühl et al. [1], which processes the frames a few at a time, as if they arrived one after another, and never stores the whole video. Alternatively, you can track the SVD of the frames themselves, one frame after another, by treating the change between consecutive frames as a small modification of the previous frame (Approach B below). You do **not** need to implement the SVD itself: use `numpy.linalg.svd` [4] wherever an SVD is needed.

Background

Convert each frame to grayscale and reshape it into one long column vector of N pixel values. Placing the T frames side by side gives the matrix $Y \in \mathbb{R}^{N \times T}$, one column per frame, with usually $N \gg T$. Its (reduced) SVD is

$$Y = USV^{\mathsf{T}},$$

where U and V have orthonormal columns and S is diagonal with the singular values $s_1 \geq s_2 \geq \cdots \geq 0$. The columns of U are basis images, and each frame is a combination of them. Keeping only the q largest singular values and the corresponding columns of U and V gives the best approximation of Y of rank q. If the singular values decay quickly, a small q already reproduces the video well, and storing the truncated U, S and V needs much less memory than storing all frames.

For a long video, Y may not fit in memory. The incremental SVD [1, 2, 3] instead builds the approximation step by step. You may choose one of the following two approaches.

Approach A: adding frames as new columns. Suppose we already have U, S and V for the frames seen so far, and a new block $B \in \mathbb{R}^{N \times b}$ of b frames arrives. Following [1, Section 2.2]:

1. Project the new frames onto the current basis: $M = U^{\mathsf{T}}B$.
2. Compute the part that is new, $P = B - UM$, and its QR decomposition $P = Q_P R_P$.
3. Form the small matrix
$$K = \begin{bmatrix} S & M \\ 0 & R_P \end{bmatrix}$$
and compute its SVD, $K = U'S'V'^{\mathsf{T}}$.
4. Update the factors:
$$U \leftarrow [\,U \;\; Q_P\,]\,U', \qquad S \leftarrow S', \qquad V \leftarrow \begin{bmatrix} V & 0 \\ 0 & I \end{bmatrix} V'.$$
5. Truncate: keep only the q largest singular values and the corresponding columns of U and V.

Only the small matrix K is decomposed, so each update is cheap. In exact arithmetic, and without truncation, the result is the same as the SVD of all frames at once. In floating-point arithmetic the columns of U slowly lose their orthogonality, especially when P is nearly zero, so in practice one re-orthogonalizes U from time to time [1].

Approach B: updating one frame to the next. Here each frame is kept as an image matrix F_k (pixels in rows and columns). Start from the SVD of the first frame, $F_1 = USV^{\mathsf{T}}$. The next frame is $F_{k+1} = F_k + D_k$, where the difference $D_k = F_{k+1} - F_k$ is usually small. Write (or approximate) the difference as a product $D_k \approx BC^{\mathsf{T}}$ with only b columns in B and C, for

example from the b largest singular values of D_k. Then update the SVD with the general formula for $Y \to Y + BC^\mathsf{T}$ in [1, Section 2.2]:

1. Compute $M_B = U^\mathsf{T}B$ and the QR decomposition $B - UM_B = Q_B R_B$, and likewise $M_C = V^\mathsf{T}C$ and $C - VM_C = Q_C R_C$.
2. Form the small matrix
$$K = \begin{bmatrix} S & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} M_B \\ R_B \end{bmatrix} \begin{bmatrix} M_C \\ R_C \end{bmatrix}^\mathsf{T}$$
and compute its SVD, $K = U'S'V'^\mathsf{T}$.
3. Update: $U \leftarrow [\,U \;\; Q_B\,]\,U'$, $S \leftarrow S'$, $V \leftarrow [\,V \;\; Q_C\,]\,V'$, and truncate to rank q.

This gives a low-rank approximation of every frame without computing a new SVD of each frame. With truncation, the small errors made in each step can add up from frame to frame.

Tasks for Project 2 (report, due November 20, 2026)

1. Choose a short video and use at most 100 frames. Read its frames into Python (for example with the `imageio` package), convert them to grayscale, make them small (for example 100×100 pixels), and build the matrix Y.
2. Compute the SVD of Y with `numpy.linalg.svd` [4] and plot the singular values (log scale).
3. Implement the incremental SVD with Approach A or Approach B above, with optional truncation to rank q. In Approach A, process the frames in blocks of b frames (for example $b = 10$). Use `numpy.linalg.svd` and `numpy.linalg.qr` for the small decompositions.
4. Run it *without* truncation and check your result against `numpy.linalg.svd`: in Approach A, compare the singular values with those of the full SVD of Y; in Approach B, compare the singular values of a few frames with the SVD of those frames. (If they differ, the columns of U have probably lost their orthogonality; see the remark above.)
5. **(Bonus)** Check how orthogonal the columns of U stay by computing $\|U^\mathsf{T}U - I\|_F$ after each update, and add a re-orthogonalization step if needed.
6. **(Bonus)** Implement both approaches.

Tasks for the final project (poster, due December 11, 2026)

1. Compare rank-q approximations from the incremental SVD (with truncation) with those from `numpy.linalg.svd`, for a few values of q, for example $q = 1, 5, 20$. Show one original frame next to its reconstructions, and plot the relative error against q (Approach A: $\|Y - Y_q\|_F / \|Y\|_F$ for the whole video; Approach B: the error of each frame, also against the frame number).
2. Discuss what you observe: is the incremental SVD as good as the full SVD? What are its benefits (for example memory) and shortcomings?
3. **(Bonus)** Study how the block size b affects the accuracy and the computing time.

Going further

These optional steps count toward the bonus points. For example, you can show the first few columns of U as images and explain what they represent.

References

[1] N. Kühl, H. Fischer, M. Hinze and T. Rung (2023). An incremental singular value decomposition approach for large-scale spatially parallel & distributed but temporally serial data – applied to technical flows. arXiv:2302.09149. https://arxiv.org/abs/2302.09149

[2] M. Brand (2002). Incremental singular value decomposition of uncertain data with missing values. In *Computer Vision – ECCV 2002*, Lecture Notes in Computer Science, vol. 2350, Springer. https://doi.org/10.1007/3-540-47969-4_47

[3] M. Brand (2006). Fast low-rank modifications of the thin singular value decomposition. *Linear Algebra and its Applications*, 415, 20–30. https://www.merl.com/publications/TR2006-059

[4] NumPy. *numpy.linalg.svd*. https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html